Cimple

Language Documentation

Cimple is a beginner-friendly programming language with a C-like syntax. It is designed to make learning to program easier, without having to worry about complex concepts such as pointers and manual memory management.

1. Data Types

A data type tells the language what kind of value a variable holds.

Keyword	Description	Example
`integer`	Whole number (32-bit)	`integer x = 5;`
`double`	Decimal number (64-bit)	`double pi = 3.14;`
`string`	Text (sequence of characters)	`string name = "Anna";`
`boolean`	True/false value	`boolean ok = true;`
`void`	No return value (functions only)	`void myFunction() {...}`

2. Variables

Variables are declared with a data type, a name, and an initial value:

```
integer age = 17;
string message = "Hello world";
double price = 49.95;
boolean active = true;
boolean compare = 5 < 7;
```

A variable's value can be reassigned after declaration:

```
age = 18;
```

3. Operators

3.1 Arithmetic

Operator	Description	Example
`+`	Addition	`integer sum = 3 + 2;`
`-`	Subtraction	`integer dif = 5 - 1;`

Operator	Description	Example
*	Multiplication	integer prod = 4 * 2;
/	Division	double q = 10 / 4;
MOD	Modulus (remainder)	integer r = 10 MOD 3;

3.2 Comparison

Operator	Description	Example
==	Equal to	boolean eq = 5 == 7;
!=	Not equal to	boolean noteq = 8 != 5;
<	Less than	boolean less = 6 < 4;
>	Greater than	boolean great = 8 > 2;
<=	Less than or equal to	boolean leq = 5 <= 5;
>=	Greater than or equal to	boolean geq = 6 >= 1;

3.3 Logical Operators

Operator	Description	Example
AND	Logical and	boolean a = true AND false
OR	Logical or	boolean b = true OR true
!	Negation	boolean c = !true

4. Control Structures

4.1 If / else

Executes code based on a condition:

```
if(x > 10) {
    print("Large number");
} else {
    print("Small number");
}
```

4.2 While loop

Repeats a block of code as long as the condition is true:

```
integer i = 0;
while(i < 5) {
    print(i);
    i = i + 1;
}
```

5. Arrays

An array stores multiple values of the same type.

Declare with values:

```
integer numbers[] = [1, 2, 3, 4, 5];
```

Declare an empty array with a fixed size:

```
integer numbers[10];
```

Access or set a value:

```
numbers[0] = 99;
print(numbers[0]);
```

> *The first element always has index 0.*

6. Functions

A function is defined with a return type, a name, parameters, and a return expression:

```
integer addTwo(integer a, integer b) {
    return a + b;
}
```

A function with no return value uses void:

```
void sayHello(string name) {
    print("Hello, " + name);
    return 0;
}
```

Call a function like this:

```
integer result = addTwo(3, 4);
sayHello("Maria");
```

7. Print

print is used to display output in the terminal:

```
void main() {
    print("Test program”);
    return;
```

```
}
```

8. Comments

Comments explain code and are ignored by the language:

```
/* This is a comment */

/*
   This is a
   multi-line comment
*/
```

9. A Complete Example

```
integer factorial(integer n) {
    integer result = 1;
    while(n > 1) {
        result = result * n;
        n = n - 1;
    }
    return result;
}

void main(){
   integer answer = factorial(5);
   print("5! = ", answer);
   return;
}
```

10. Operator Precedence

Operators are evaluated in this order (high to low):

Precedence	Operator(s)
1 (highest)	( )
2	!
3	*, /, MOD
4	+, -
5	<, <=, >, >=
6	==, !=
7	AND, OR
8 (lowest)	= (assignment)

11. Rules to Remember

- All statements end with ;
- Code blocks are enclosed in { }
- true and false are written in lowercase
- The language is case-sensitive — x and X are different variables
- Division by 0 is not allowed and will terminate the program
- A variable must be declared before it is used
- All elements in an array must be of the same type